МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ
«НОВОСИБИРСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

Кафедра защиты информации



ЛАБОРАТОРНАЯ РАБОТА №2

по дисциплине: «Безопасность систем баз данных»

Выполнил:

Студент гр. «АБс-322», «АВТФ»

Бутаков Никита Максимович

«___» _____ 2025г

_____

(подпись)

Проверил:

Ассистент кафедры ЗИ

Питько Яна Андреевна

«___» _____ 2025г

_____

(подпись)

Новосибирск 2025

Содержание

Введение 3
1. SECURITY DEFINER-функции 4
2. Тесты прав 8
3. CHECK vs триггер 10
4. Журнал вызовов для SECURITY DEFINER-функций 12
Вывод 13
Приложение А 14
Приложение Б 25
Приложение В 37
Приложение Г 46

Введение

Цель работы: реализовать безопасные операции через функции с повышенными привилегиями, сравнить методы контроля бизнес-правил.

Тема: «Защита БД студий дубляжа – управление доступом к авторским материалам, контрактам с актёрами, расписаниям записей и защищённым архивам аудиофайлов».

Задачи:

1. SECURITY DEFINER-функции (точечное повышение): реализуйте 2–3 функции для чувствительных операций в вашей предметной области (например, создание ключевого объекта, обновление поля с ограничениями, регистрация «транзакции»). SECURITY DEFINER, валидация входа, понятные сообщения об ошибках. Функции должны работать под правами владельца, но вызываться из-под ролей с ограниченными правами.

2. Тесты прав (≥ 8 кейсов): позитив/негатив: чтение разрешённых таблиц, попытки выполнения DDL не админом, DML в audit, вызовы SECURITY DEFINER с валидными/не валидными данными. psql-скрипты или pgTAP.

3. CHECK vs триггер: выберите сложное бизнес-правило из вашей области (например, end_date ≥ start_date, сумма ≥ 0, согласованность статуса). Сравните CHECK и триггер: корректность, удобство аудита, производительность на вставке 10 тыс. строк. Зафиксируйте выбранный подход.

4. Журнал вызовов для SECURITY DEFINER-функций: добавьте таблицу audit.function_calls (call_time, function_name, caller_role, input_params, success). Модифицируйте ваши функции так, чтобы они логировали каждый вызов. Протестируйте и покажите лог после нескольких вызовов.

1. SECURITY DEFINER-функции

В качестве чувствительных операций, для которых были реализованы SECURITY DEFINER-функции, были выбраны:

1. Создание контракта с актёром (app.create_contract) – операция создания ключевого бизнес-объекта, содержащего конфиденциальные данные (сумма контракта, условия). Требует повышенных привилегий для проверки целостности данных и логирования.

2. Обновление зарплаты в контракте (app.update_contract_salary) – операция изменения конфиденциальных финансовых данных. Требует строгой валидации и контроля доступа для предотвращения несанкционированных изменений.

3. Регистрация сессии записи (app.register_recording_session) – бизнес-транзакция, регистрирующая сессию записи дубляжа. Требует проверки статуса контракта и валидации параметров сессии для обеспечения целостности бизнес-процессов.

Все функции реализованы с использованием модификатора SECURITY DEFINER, что позволяет выполнять их с правами владельца функции (роль app_owner), при этом вызывать их могут пользователи с ограниченными правами. Это обеспечивает принцип наименьших привилегий: пользователи получают доступ только к необходимым операциям, не имея прямых прав на изменение таблиц.

Также в каждой из функций реализована валидация данных для корректной вставки. Соотношение таблицы и прописанных в ней правил представлено в таблице 1.

Таблица 1 – Валидация данных в функциях

Название функции	**Проверяемые данные**	**Описание проверки**
app.create_contract	p_project_id	Проверка существования проекта в таблице app.projects
	p_actor_id	Проверка существования и активности актёра в таблице app.actors (поле is_active = true)
	p_contract_number	Проверка уникальности номера контракта в таблице app.contracts
	p_contract_number	Проверка обязательного поля: не NULL и не пустая строка
	p_role_description	Проверка обязательного поля: не NULL и не пустая строка
	p_salary_amount	Проверка, что сумма контракта не NULL и больше 0
	p_signed_date, p_end_date	Проверка корректности дат: end_date >= signed_date (если обе даты указаны)
app.update_contract_salary	p_contract_id	Проверка существования контракта в таблице app.contracts

Продолжение таблицы 1

Название функции	**Проверяемые данные**	**Описание проверки**
	p_new_salary_amount	Проверка, что новая сумма не NULL и больше 0
	p_new_salary_amount, v_old_salary	Проверка, что новая сумма отличается от старой
	Статус контракта	Проверка, что контракт не завершён (status NOT IN ('cancelled', 'completed'))
app.register_recording_session	p_contract_id	Проверка существования контракта в таблице app.contracts
	Статус контракта	Проверка, что контракт активен (status = 'active')
	p_session_date	Проверка, что дата сессии не в прошлом более чем на 1 день
	p_duration_hours	Проверка, что длительность сессии в диапазоне от 0 до 24 часов

Продолжение таблицы 1

Название функции	**Проверяемые данные**	**Описание проверки**
	p_studio_location	Проверка обязательного поля: не NULL и не пустая строка

Права на выполнение функций были предоставлены app_owner, app_writer и dml_admin. Исходный код для SECURITY DEFINER-функций представлен в приложении А.

2. Тесты прав

Написанные тесты работают по принципу присваивания тестовому пользователю проверяемой роли и дальнейшая попытка выполнения различных команд.

Всего есть два возможных варианта выполнения теста: либо операция проходит успешно, либо пользователю отказывается в доступе. Если ожидаемый результат совпадает с действительным, то на экран выводится сообщение об успехе.

В конце тестирования происходит очистка тестовых данных для того, чтобы не оставлять в базе лишних записей.

Демонстрация работы сценариев проверки, а также скрипты для тестирования работоспособности созданных SECURITY DEFINER функций представлены на рисунках 2 и 3.

```
=== КЕЙС 1: Успешное создание контракта (позитивный) ===
DO
SET
psql:/tmp/lr2_04_tests.sql:163: NOTICE:  ✅ Контракт успешно создан с ID: 14
DO
RESET

=== КЕЙС 2: Ошибка валидации – несуществующий проект (негативный) ===
SET
psql:/tmp/lr2_04_tests.sql:204: NOTICE:  ✅ Ожидаемая ошибка: Проект с ID 99999 не существует
DO
RESET

=== КЕЙС 3: Ошибка валидации – отрицательная зарплата (негативный) ===
SET
psql:/tmp/lr2_04_tests.sql:247: NOTICE:  ✅ Ожидаемая ошибка: Сумма контракта должна быть положительным числом
DO
RESET

=== КЕЙС 4: Успешное обновление зарплаты (позитивный) ===
DO
SET
psql:/tmp/lr2_04_tests.sql:298: NOTICE:  ✅ Зарплата успешно обновлена для контракта ID: 1
DO
RESET

=== КЕЙС 5: Попытка DDL неадмином (негативный) ===
SET
psql:/tmp/lr2_04_tests.sql:326: NOTICE:  ✅ Ожидаемая ошибка: Недостаточно прав для DDL операции
DO
RESET

=== КЕЙС 6: Попытка DML в audit (негативный) ===
SET
DO
psql:/tmp/lr2_04_tests.sql:352: NOTICE:  ✅ Ожидаемая ошибка: Недостаточно прав для DML в audit
RESET

=== КЕЙС 7: Чтение разрешённых таблиц (позитивный) ===
DO
SET
psql:/tmp/lr2_04_tests.sql:400: NOTICE:  ✅ Актёров доступно (с current_actor_id): 1
psql:/tmp/lr2_04_tests.sql:400: NOTICE:  ✅ Проектов доступно: 2
psql:/tmp/lr2_04_tests.sql:400: NOTICE:  ✅ Языков доступно: 8
DO
RESET
```

Рисунок 2 – Демонстрация сценариев проверки доступа

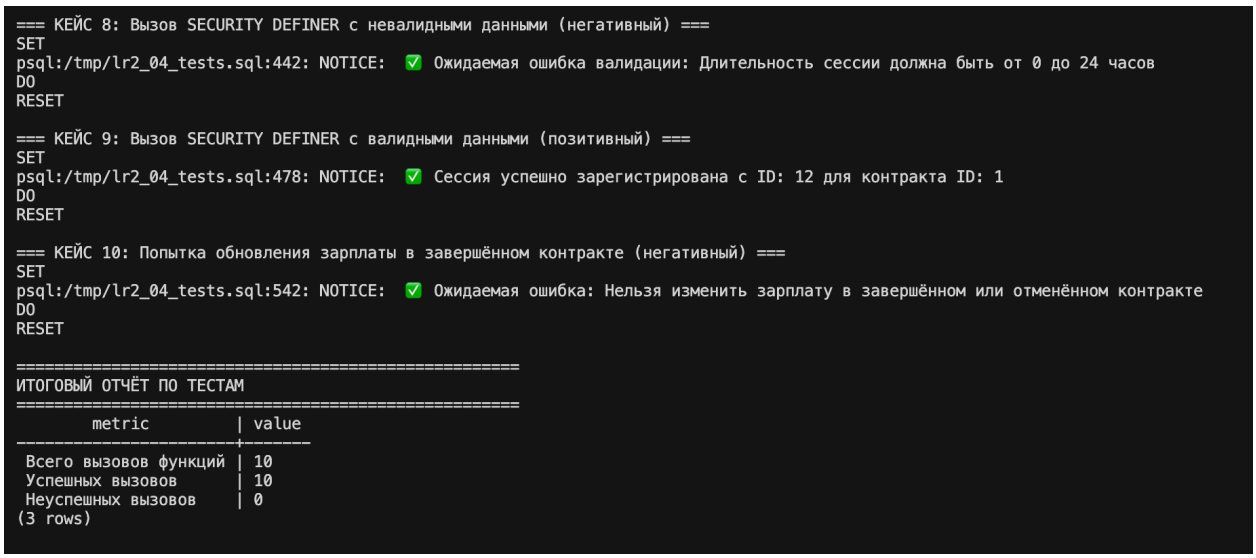


Рисунок 3 – Демонстрация сценариев проверки доступа

Исходный код для тестов прав (10 кейсов) представлен в приложении Б.

3. CHECK vs триггер

Для сравнения реализации через CHECK-ограничения и триггеры было выбрано следующее бизнес-правило для таблицы контрактов:

1. Правило валидации дат контракта: дата окончания контракта (end_date) должна быть больше или равна дате подписания (signed_date). Это обеспечивает логическую целостность данных, исключая ситуацию, когда контракт заканчивается раньше, чем был подписан.

2. Правило для завершённых контрактов: если контракт имеет статус completed, то дата окончания (end_date) должна быть установлена и не может быть в будущем (т.е. end_date <= CURRENT_DATE). Это гарантирует, что завершённый контракт не может иметь дату окончания в будущем.

CHECK-ограничения были реализованы на тестовой таблице app.contracts_check_test. Для обеспечения валидации дат контракта было создано два ограничения.

Для сравнения была создана тестовая таблица app.contracts_trigger_test без CHECK-ограничений, а валидация реализована через триггерную функцию app.validate_contract_dates(), которая выполняется перед вставкой или обновлением строки (BEFORE INSERT OR UPDATE).

Для сравнения производительности была разработана функция app.test_performance_check_vs_trigger(), которая выполняет вставку 10,000 записей с валидными данными в обе тестовые таблицы и измеряет время выполнения.

Результаты тестирования на 10,000 строк представлен на рисунке 4.

```
================================================
ТЕСТ ПРОИЗВОДИТЕЛЬНОСТИ: CHECK vs TRIGGER
================================================

Запуск теста на 10,000 строк...
Это может занять некоторое время...
 method  | inserted_count | failed_count | total_time_ms | avg_time_per_insert_ms
---------+----------------+--------------+---------------+------------------------
 CHECK   |           9864 |          136 |    133.895000 | 0.0135741078669910786 7
 TRIGGER |           9899 |          101 |     97.713000 | 0.0098709970704111526 4
(2 rows)
```

Рисунок 4 – Сравнение CHECK и TRIGGER

Исходя из результатов тестирования производительности, при выполнении операции вставки 10,000 строк с одинаковыми данными были получены следующие показатели:

1. Для метода CHECK-ограничений было успешно вставлено 9,864 записи, при этом 136 операций вставки завершились неудачей. Общее время выполнения составило 133.895 миллисекунд.

2. Для метода триггеров было успешно вставлено 9,899 записей, при этом 101 операции вставки завершились неудачей. Общее время выполнения составило 97.713 миллисекунд.

Исходный код для теста производительности представлен в приложении В.

4. Журнал вызовов для SECURITY DEFINER-функций

Журнал вызовов реализован в виде таблицы audit.function_calls. Таблица содержит следующие поля для полного отслеживания вызовов функций:

- call_time – точное время вызова функции с учётом временной зоны
- function_name – имя вызванной SECURITY DEFINER-функции
- caller_role – роль пользователя, от имени которой была вызвана функция
- input_params – входные параметры функции в формате JSON, что позволяет хранить сложные структуры данных
- success – булево значение, указывающее на успешность выполнения функции

Все три SECURITY DEFINER-функции, реализованные в рамках задания, модифицированы для автоматического логирования каждого вызова.

Результат работы записи в таблицу аудита после нескольких вызовов функций представлен на рисунке 5.

	call_id [PK] bigint	call_time timestamp with time zone	function_name character varying (255)	caller_role character varying (100)	input_p jsonb	success boolean
1	1	2025-11-07 15:20:09.449035+...	create_contract	studio_manager	{"ac...	true
2	6	2025-11-07 15:20:09.453681+...	update_contract_salary	studio_manager	{"re...	true
3	9	2025-11-07 15:20:09.457737+...	register_recording_sessi...	studio_manager	{"co...	true
4	10	2025-11-07 15:20:09.459332+...	create_contract	studio_manager	{"ac...	true
5	13	2025-11-07 15:20:09.533792+...	create_contract	studio_manager	{"ac...	true
6	14	2025-11-07 15:20:09.537253+...	update_contract_salary	studio_manager	{"re...	true

Рисунок 5 – Запись вызова функций в таблицу аудита

Исходный код для создания журнала вызовов и модификации функции представлен в приложении Г.

Вывод

В ходе лабораторной работы была разработана система управления данными, обеспечивающая безопасное и целостное выполнение операций. Для защиты ключевых бизнес-процессов были применены функции с правами создателя (SECURITY DEFINER), оснащенные многоуровневой валидацией входных данных для гарантии соответствия бизнес-требованиям.

Эффективность ролевой модели была подтверждена тестированием, показавшим строгое разграничение прав и блокировку несанкционированных действий. В области целостности данных было установлено, что для простых правил валидации ограничения CHECK предпочтительнее триггеров.

Для аудита критически важных операций создана таблица function_calls, регистрирующая параметры вызовов. Архитектура логирования учитывает специфику PostgreSQL, в частности отсутствие автономных транзакций, и реализует раздельное логирование: успешные операции фиксируются в СУБД, а ошибки обрабатываются на уровне приложения.

Приложение А

```
-- ======================================================
-- SECURITY DEFINER-функции для чувствительных операций
-- ======================================================

-- Устанавливаем роль для создания функций
SET ROLE app_owner;

-- ======================================================
-- ФУНКЦИЯ 1: create_contract
-- Создание контракта с актёром (чувствительная операция)
-- ======================================================

CREATE OR REPLACE FUNCTION app.create_contract(
    p_project_id INTEGER,
    p_actor_id INTEGER,
    p_contract_number VARCHAR(50),
    p_role_description TEXT,
    p_salary_amount DECIMAL(10,2),
    p_currency VARCHAR(3) DEFAULT 'USD',
    p_terms TEXT DEFAULT NULL,
    p_signed_date DATE DEFAULT CURRENT_DATE,
    p_end_date DATE DEFAULT NULL
)
RETURNS INTEGER
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = app, ref, audit, public
AS $$
DECLARE
    v_start_time TIMESTAMPTZ;
    v_end_time TIMESTAMPTZ;
    v_execution_time NUMERIC;
    v_contract_id INTEGER;
    v_input_params JSONB;
    v_return_value JSONB;
BEGIN
    -- Засекаем время выполнения
    v_start_time := clock_timestamp();

    -- Формируем JSON с параметрами для логирования
    v_input_params := jsonb_build_object(
        'project_id', p_project_id,
        'actor_id', p_actor_id,
        'contract_number', p_contract_number,
        'role_description', p_role_description,
        'salary_amount', p_salary_amount,
        'currency', p_currency
    );

    -- ВАЛИДАЦИЯ ВХОДНЫХ ДАННЫХ
```

```
    -- Проверка существования проекта
    IF NOT EXISTS (SELECT 1 FROM app.projects WHERE project_id = p_project_id) THEN
        PERFORM audit.log_function_call(
            'app', 'create_contract',
            v_input_params,
            NULL,
            false,
            'ОШИБКА: Проект с ID ' || p_project_id || ' не существует',
            NULL
        );
        RAISE EXCEPTION 'Проект с ID % не существует', p_project_id;
    END IF;

    -- Проверка существования актёра
    IF NOT EXISTS (SELECT 1 FROM app.actors WHERE actor_id = p_actor_id AND is_active
= true) THEN
        PERFORM audit.log_function_call(
            'app', 'create_contract',
            v_input_params,
            NULL,
            false,
            'ОШИБКА: Актёр с ID ' || p_actor_id || ' не существует или неактивен',
            NULL
        );
        RAISE EXCEPTION 'Актёр с ID % не существует или неактивен', p_actor_id;
    END IF;

    -- Проверка уникальности номера контракта
    IF EXISTS (SELECT 1 FROM app.contracts WHERE contract_number = p_contract_number)
THEN
        PERFORM audit.log_function_call(
            'app', 'create_contract',
            v_input_params,
            NULL,
            false,
            'ОШИБКА: Контракт с номером ' || p_contract_number || ' уже существует',
            NULL
        );
        RAISE EXCEPTION 'Контракт с номером % уже существует', p_contract_number;
    END IF;

    -- Проверка суммы контракта
    IF p_salary_amount IS NULL OR p_salary_amount <= 0 THEN
        PERFORM audit.log_function_call(
            'app', 'create_contract',
            v_input_params,
            NULL,
            false,
            'ОШИБКА: Сумма контракта должна быть положительным числом',
            NULL
        );
        RAISE EXCEPTION 'Сумма контракта должна быть положительным числом';
    END IF;
```

```
    -- Проверка дат (end_date должен быть после signed_date)
    IF p_end_date IS NOT NULL AND p_signed_date IS NOT NULL AND p_end_date <
p_signed_date THEN
        PERFORM audit.log_function_call(
            'app', 'create_contract',
            v_input_params,
            NULL,
            false,
            'ОШИБКА: Дата окончания контракта не может быть раньше даты подписания',
            NULL
        );
        RAISE EXCEPTION 'Дата окончания контракта не может быть раньше даты
подписания';
    END IF;

    -- Проверка обязательных полей
    IF p_contract_number IS NULL OR trim(p_contract_number) = '' THEN
        PERFORM audit.log_function_call(
            'app', 'create_contract',
            v_input_params,
            NULL,
            false,
            'ОШИБКА: Номер контракта обязателен для заполнения',
            NULL
        );
        RAISE EXCEPTION 'Номер контракта обязателен для заполнения';
    END IF;

    IF p_role_description IS NULL OR trim(p_role_description) = '' THEN
        PERFORM audit.log_function_call(
            'app', 'create_contract',
            v_input_params,
            NULL,
            false,
            'ОШИБКА: Описание роли обязательно для заполнения',
            NULL
        );
        RAISE EXCEPTION 'Описание роли обязательно для заполнения';
    END IF;

    -- СОЗДАНИЕ КОНТРАКТА
    INSERT INTO app.contracts (
        project_id,
        actor_id,
        contract_number,
        role_description,
        salary_amount,
        currency,
        terms,
        signed_date,
        end_date,
        status
```

```
    ) VALUES (
        p_project_id,
        p_actor_id,
        p_contract_number,
        p_role_description,
        p_salary_amount,
        p_currency,
        p_terms,
        p_signed_date,
        p_end_date,
        'active'
    )
    RETURNING contract_id INTO v_contract_id;

    -- Формируем возвращаемое значение для логирования
    v_return_value := jsonb_build_object('contract_id', v_contract_id);

    -- Вычисляем время выполнения
    v_end_time := clock_timestamp();
    v_execution_time := EXTRACT(EPOCH FROM (v_end_time - v_start_time)) * 1000;

    -- ЛОГИРОВАНИЕ УСПЕШНОГО ВЫЗОВА
    PERFORM audit.log_function_call(
        'app', 'create_contract',
        v_input_params,
        v_return_value,
        true,
        NULL,
        v_execution_time
    );

    RETURN v_contract_id;

EXCEPTION
    WHEN OTHERS THEN
        -- Вычисляем время выполнения при ошибке
        v_end_time := clock_timestamp();
        v_execution_time := EXTRACT(EPOCH FROM (v_end_time - v_start_time)) * 1000;

        -- ЛОГИРОВАНИЕ ОШИБКИ
        PERFORM audit.log_function_call(
            'app', 'create_contract',
            v_input_params,
            NULL,
            false,
            'ОШИБКА: ' || SQLERRM,
            v_execution_time
        );

        -- Пробрасываем ошибку дальше
        RAISE;
END;
$$;
```

```sql
-- ======================================================
-- ФУНКЦИЯ 2: update_contract_salary
-- Обновление зарплаты в контракте (конфиденциальные данные)
-- ======================================================

CREATE OR REPLACE FUNCTION app.update_contract_salary(
    p_contract_id INTEGER,
    p_new_salary_amount DECIMAL(10,2),
    p_reason TEXT DEFAULT NULL
)
RETURNS BOOLEAN
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = app, ref, audit, public
AS $$
DECLARE
    v_start_time TIMESTAMPTZ;
    v_end_time TIMESTAMPTZ;
    v_execution_time NUMERIC;
    v_old_salary DECIMAL(10,2);
    v_input_params JSONB;
    v_return_value JSONB;
BEGIN
    v_start_time := clock_timestamp();

    -- Формируем JSON с параметрами
    v_input_params := jsonb_build_object(
        'contract_id', p_contract_id,
        'new_salary_amount', p_new_salary_amount,
        'reason', p_reason
    );

    -- ВАЛИДАЦИЯ

    -- Проверка существования контракта
    SELECT salary_amount INTO v_old_salary
    FROM app.contracts
    WHERE contract_id = p_contract_id;

    IF NOT FOUND THEN
        PERFORM audit.log_function_call(
            'app', 'update_contract_salary',
            v_input_params,
            NULL,
            false,
            'ОШИБКА: Контракт с ID ' || p_contract_id || ' не найден',
            NULL
        );
        RAISE EXCEPTION 'Контракт с ID % не найден', p_contract_id;
    END IF;

    -- Проверка суммы
```

```
    IF p_new_salary_amount IS NULL OR p_new_salary_amount <= 0 THEN
        PERFORM audit.log_function_call(
            'app', 'update_contract_salary',
            v_input_params,
            NULL,
            false,
            'ОШИБКА: Новая сумма должна быть положительным числом',
            NULL
        );
        RAISE EXCEPTION 'Новая сумма должна быть положительным числом';
    END IF;

    -- Проверка, что сумма изменилась
    IF p_new_salary_amount = v_old_salary THEN
        PERFORM audit.log_function_call(
            'app', 'update_contract_salary',
            v_input_params,
            NULL,
            false,
            'ОШИБКА: Новая сумма совпадает со старой',
            NULL
        );
        RAISE EXCEPTION 'Новая сумма совпадает со старой (%.2f)', v_old_salary;
    END IF;

    -- Проверка статуса контракта
    IF EXISTS (
        SELECT 1 FROM app.contracts
        WHERE contract_id = p_contract_id
        AND status IN ('cancelled', 'completed')
    ) THEN
        PERFORM audit.log_function_call(
            'app', 'update_contract_salary',
            v_input_params,
            NULL,
            false,
            'ОШИБКА: Нельзя изменить зарплату в завершённом или отменённом контракте',
            NULL
        );
        RAISE EXCEPTION 'Нельзя изменить зарплату в завершённом или отменённом
контракте';
    END IF;

    -- ОБНОВЛЕНИЕ ЗАРПЛАТЫ
    UPDATE app.contracts
    SET
        salary_amount = p_new_salary_amount
    WHERE contract_id = p_contract_id;

    -- Формируем возвращаемое значение
    v_return_value := jsonb_build_object(
        'contract_id', p_contract_id,
        'old_salary', v_old_salary,
```

```
        'new_salary', p_new_salary_amount,
        'updated', true
    );

    -- Вычисляем время выполнения
    v_end_time := clock_timestamp();
    v_execution_time := EXTRACT(EPOCH FROM (v_end_time - v_start_time)) * 1000;

    -- ЛОГИРОВАНИЕ
    PERFORM audit.log_function_call(
        'app', 'update_contract_salary',
        v_input_params,
        v_return_value,
        true,
        NULL,
        v_execution_time
    );

    RETURN true;

EXCEPTION
    WHEN OTHERS THEN
        v_end_time := clock_timestamp();
        v_execution_time := EXTRACT(EPOCH FROM (v_end_time - v_start_time)) * 1000;

        PERFORM audit.log_function_call(
            'app', 'update_contract_salary',
            v_input_params,
            NULL,
            false,
            'ОШИБКА: ' || SQLERRM,
            v_execution_time
        );

        RAISE;
END;
$$;

-- ======================================================
-- ФУНКЦИЯ 3: register_recording_session
-- Регистрация сессии записи (бизнес-транзакция)
-- ======================================================

CREATE OR REPLACE FUNCTION app.register_recording_session(
    p_contract_id INTEGER,
    p_session_date DATE,
    p_session_time TIME,
    p_studio_location VARCHAR(100),
    p_duration_hours DECIMAL(4,2),
    p_notes TEXT DEFAULT NULL
)
RETURNS INTEGER
LANGUAGE plpgsql
```

```
SECURITY DEFINER
SET search_path = app, ref, audit, public
AS $$
DECLARE
    v_start_time TIMESTAMPTZ;
    v_end_time TIMESTAMPTZ;
    v_execution_time NUMERIC;
    v_session_id INTEGER;
    v_contract_status VARCHAR(20);
    v_input_params JSONB;
    v_return_value JSONB;
BEGIN
    v_start_time := clock_timestamp();

    -- Формируем JSON с параметрами
    v_input_params := jsonb_build_object(
        'contract_id', p_contract_id,
        'session_date', p_session_date,
        'session_time', p_session_time,
        'studio_location', p_studio_location,
        'duration_hours', p_duration_hours
    );

    -- ВАЛИДАЦИЯ

    -- Проверка существования контракта
    SELECT status INTO v_contract_status
    FROM app.contracts
    WHERE contract_id = p_contract_id;

    IF NOT FOUND THEN
        PERFORM audit.log_function_call(
            'app', 'register_recording_session',
            v_input_params,
            NULL,
            false,
            'ОШИБКА: Контракт с ID ' || p_contract_id || ' не найден',
            NULL
        );
        RAISE EXCEPTION 'Контракт с ID % не найден', p_contract_id;
    END IF;

    -- Проверка статуса контракта
    IF v_contract_status != 'active' THEN
        PERFORM audit.log_function_call(
            'app', 'register_recording_session',
            v_input_params,
            NULL,
            false,
            'ОШИБКА: Контракт должен быть активным (текущий статус: ' ||
v_contract_status || ')',
            NULL
        );
```

```
        RAISE EXCEPTION 'Контракт должен быть активным (текущий статус: %)',
v_contract_status;
    END IF;

    -- Проверка даты сессии (не должна быть в прошлом более чем на 1 день)
    IF p_session_date < CURRENT_DATE - INTERVAL '1 day' THEN
        PERFORM audit.log_function_call(
            'app', 'register_recording_session',
            v_input_params,
            NULL,
            false,
            'ОШИБКА: Дата сессии не может быть в прошлом более чем на 1 день',
            NULL
        );
        RAISE EXCEPTION 'Дата сессии не может быть в прошлом более чем на 1 день';
    END IF;

    -- Проверка длительности
    IF p_duration_hours IS NULL OR p_duration_hours <= 0 OR p_duration_hours > 24 THEN
        PERFORM audit.log_function_call(
            'app', 'register_recording_session',
            v_input_params,
            NULL,
            false,
            'ОШИБКА: Длительность сессии должна быть от 0 до 24 часов',
            NULL
        );
        RAISE EXCEPTION 'Длительность сессии должна быть от 0 до 24 часов';
    END IF;

    -- Проверка обязательных полей
    IF p_studio_location IS NULL OR trim(p_studio_location) = '' THEN
        PERFORM audit.log_function_call(
            'app', 'register_recording_session',
            v_input_params,
            NULL,
            false,
            'ОШИБКА: Место проведения сессии обязательно для заполнения',
            NULL
        );
        RAISE EXCEPTION 'Место проведения сессии обязательно для заполнения';
    END IF;

    -- СОЗДАНИЕ СЕССИИ
    INSERT INTO app.recording_sessions (
        contract_id,
        session_date,
        session_time,
        studio_location,
        duration_hours,
        notes,
        status
    ) VALUES (
```

```
        p_contract_id,
        p_session_date,
        p_session_time,
        p_studio_location,
        p_duration_hours,
        p_notes,
        'scheduled'
    )
    RETURNING session_id INTO v_session_id;

    -- Формируем возвращаемое значение
    v_return_value := jsonb_build_object(
        'session_id', v_session_id,
        'contract_id', p_contract_id,
        'session_date', p_session_date,
        'status', 'scheduled'
    );

    -- Вычисляем время выполнения
    v_end_time := clock_timestamp();
    v_execution_time := EXTRACT(EPOCH FROM (v_end_time - v_start_time)) * 1000;

    -- ЛОГИРОВАНИЕ
    PERFORM audit.log_function_call(
        'app', 'register_recording_session',
        v_input_params,
        v_return_value,
        true,
        NULL,
        v_execution_time
    );

    RETURN v_session_id;

EXCEPTION
    WHEN OTHERS THEN
        v_end_time := clock_timestamp();
        v_execution_time := EXTRACT(EPOCH FROM (v_end_time - v_start_time)) * 1000;

        PERFORM audit.log_function_call(
            'app', 'register_recording_session',
            v_input_params,
            NULL,
            false,
            'ОШИБКА: ' || SQLERRM,
            v_execution_time
        );

        RAISE;
END;
$$;

-- Сбрасываем роль
```

```sql
RESET ROLE;

-- Даём доступ к функциям нужным ролям
GRANT EXECUTE ON FUNCTION app.create_contract TO app_writer, studio_manager;
GRANT EXECUTE ON FUNCTION app.update_contract_salary TO app_writer, studio_manager;
GRANT EXECUTE ON FUNCTION app.register_recording_session TO app_writer,
studio_manager;

-- Комментарии к функциям
COMMENT ON FUNCTION app.create_contract IS 'SECURITY DEFINER: Создание контракта с
актёром. Валидирует входные данные и логирует операцию';
COMMENT ON FUNCTION app.update_contract_salary IS 'SECURITY DEFINER: Обновление
зарплаты в контракте. Проверяет права и статус контракта';
COMMENT ON FUNCTION app.register_recording_session IS 'SECURITY DEFINER: Регистрация
сессии записи. Валидирует контракт и параметры сессии';

-- Уведомление
DO $$
BEGIN
    RAISE NOTICE '✅ SECURITY DEFINER-функции успешно созданы:';
    RAISE NOTICE '   - app.create_contract';
    RAISE NOTICE '   - app.update_contract_salary';
    RAISE NOTICE '   - app.register_recording_session';
END $$;
```

Приложение Б

```
-- ======================================================
-- Тесты прав
-- ======================================================

\echo '======================================================'
\echo 'ТЕСТЫ ПРАВ ДОСТУПА'
\echo '======================================================'

DROP POLICY IF EXISTS app_owner_full_access_actors ON app.actors;
CREATE POLICY app_owner_full_access_actors ON app.actors
    FOR ALL
    TO app_owner
    USING (true)
    WITH CHECK (true);

DROP POLICY IF EXISTS app_owner_full_access_contracts ON app.contracts;
CREATE POLICY app_owner_full_access_contracts ON app.contracts
    FOR ALL
    TO app_owner
    USING (true)
    WITH CHECK (true);

DROP POLICY IF EXISTS app_owner_full_access_projects ON app.projects;
CREATE POLICY app_owner_full_access_projects ON app.projects
    FOR ALL
    TO app_owner
    USING (true)
    WITH CHECK (true);

DROP POLICY IF EXISTS app_owner_full_access_sessions ON app.recording_sessions;
CREATE POLICY app_owner_full_access_sessions ON app.recording_sessions
    FOR ALL
    TO app_owner
    USING (true)
    WITH CHECK (true);

-- Проверка наличия функций ЛР2
DO $$
BEGIN
    IF NOT EXISTS (
        SELECT 1 FROM pg_proc p
        JOIN pg_namespace n ON p.pronamespace = n.oid
        WHERE n.nspname = 'app'
          AND p.proname = 'create_contract'
          AND p.prosecdef = true
    ) THEN
        RAISE EXCEPTION 'ОШИБКА: Функция app.create_contract не найдена! Выполните
инициализацию ЛР2: ./init_lr2.sh';
    END IF;

    IF NOT EXISTS (
```

```
        SELECT 1 FROM pg_proc p
        JOIN pg_namespace n ON p.pronamespace = n.oid
        WHERE n.nspname = 'app'
          AND p.proname = 'update_contract_salary'
          AND p.prosecdef = true
    ) THEN
        RAISE EXCEPTION 'ОШИБКА: Функция app.update_contract_salary не найдена!
Выполните инициализацию ЛР2: ./init_lr2.sh';
    END IF;

    IF NOT EXISTS (
        SELECT 1 FROM pg_proc p
        JOIN pg_namespace n ON p.pronamespace = n.oid
        WHERE n.nspname = 'app'
          AND p.proname = 'register_recording_session'
          AND p.prosecdef = true
    ) THEN
        RAISE EXCEPTION 'ОШИБКА: Функция app.register_recording_session не найдена!
Выполните инициализацию ЛР2: ./init_lr2.sh';
    END IF;

    RAISE NOTICE '✅ Все функции ЛР2 найдены';
END $$;

-- Проверка наличия данных (от имени postgres для обхода RLS)
DO $$
DECLARE
    v_actor_count INTEGER;
    v_project_count INTEGER;
    v_contract_count INTEGER;
BEGIN
    -- Внутри DO блока SET ROLE не работает, поэтому используем прямые запросы
    -- от имени текущего пользователя (postgres), для обхода RLS
    SELECT COUNT(*) INTO v_actor_count FROM app.actors WHERE is_active = true;
    SELECT COUNT(*) INTO v_project_count FROM app.projects;
    SELECT COUNT(*) INTO v_contract_count FROM app.contracts;

    IF v_actor_count = 0 THEN
        RAISE WARNING '⚠️  В БД нет активных актёров. Некоторые тесты могут не
пройти.';
    END IF;

    IF v_project_count = 0 THEN
        RAISE WARNING '⚠️  В БД нет проектов. Некоторые тесты могут не пройти.';
    END IF;

    IF v_contract_count = 0 THEN
        RAISE WARNING '⚠️  В БД нет контрактов. Тесты обновления контрактов могут не
пройти.';
    END IF;

    RAISE NOTICE 'Данные в БД: Актёров: %, Проектов: %, Контрактов: %', v_actor_count,
v_project_count, v_contract_count;
```

```
END $$;

-- ======================================================
-- КЕЙС 1: Успешное создание контракта через SECURITY DEFINER
-- ======================================================
\echo ''
\echo '=== КЕЙС 1: Успешное создание контракта (позитивный) ==='

-- Получаем ID для тестов (от имени postgres, который обходит RLS)
DO $$
DECLARE
    v_test_project_id INTEGER;
    v_test_actor_id INTEGER;
BEGIN
    -- Получаем данные от имени текущего пользователя (postgres обходит RLS)
    SELECT project_id INTO v_test_project_id FROM app.projects LIMIT 1;
    SELECT actor_id INTO v_test_actor_id FROM app.actors WHERE is_active = true LIMIT
1;

    -- Сохраняем ID в переменные сессии для использования в тестах
    PERFORM set_config('app.test_project_id', COALESCE(v_test_project_id::TEXT, '0'),
false);
    PERFORM set_config('app.test_actor_id', COALESCE(v_test_actor_id::TEXT, '0'),
false);

    IF v_test_project_id IS NULL THEN
        RAISE EXCEPTION 'ОШИБКА: В БД нет проектов. Выполните скрипты ЛР1
(script8.sql)';
    END IF;

    IF v_test_actor_id IS NULL THEN
        RAISE EXCEPTION 'ОШИБКА: В БД нет активных актёров. Выполните скрипты ЛР1
(script8.sql)';
    END IF;
END $$;

SET ROLE studio_manager;

DO $$
DECLARE
    v_contract_id INTEGER;
    v_project_id INTEGER;
    v_actor_id INTEGER;
BEGIN
    -- Получаем ID из переменных сессии
    v_project_id := current_setting('app.test_project_id', true)::INTEGER;
    v_actor_id := current_setting('app.test_actor_id', true)::INTEGER;

    -- Создаём контракт через SECURITY DEFINER функцию
    -- Используем уникальный номер контракта на основе timestamp
    BEGIN
        SELECT app.create_contract(
            p_project_id := v_project_id,
```

```
            p_actor_id := v_actor_id,
            p_contract_number := 'TEST-CONTRACT-' || to_char(clock_timestamp(),
'YYYYMMDDHH24MISS'),
            p_role_description := 'Тестовая роль для проверки',
            p_salary_amount := 30000.00,
            p_currency := 'USD',
            p_signed_date := CURRENT_DATE,
            p_end_date := (CURRENT_DATE + INTERVAL '6 months')::DATE
        ) INTO v_contract_id;

        RAISE NOTICE '✅ Контракт успешно создан с ID: %', v_contract_id;
    EXCEPTION
        WHEN unique_violation THEN
            RAISE NOTICE '⚠️  Контракт с таким номером уже существует, пропускаем
тест';
        WHEN OTHERS THEN
            RAISE;
    END;
END $$;

RESET ROLE;

-- =====================================================
-- КЕЙС 2: Ошибка валидации - несуществующий проект (негативный)
-- =====================================================
\echo ''
\echo '=== КЕЙС 2: Ошибка валидации - несуществующий проект (негативный) ==='

SET ROLE studio_manager;

DO $$
DECLARE
    v_contract_id INTEGER;
    v_error TEXT;
    v_actor_id INTEGER;
BEGIN
    -- Получаем ID актёра из переменных сессии
    v_actor_id := current_setting('app.test_actor_id', true)::INTEGER;

    IF v_actor_id IS NULL OR v_actor_id = 0 THEN
        RAISE WARNING '⚠️  В БД нет активных актёров. Пропускаем тест.';
        RETURN;
    END IF;

    BEGIN
        SELECT app.create_contract(
            p_project_id := 99999, -- Несуществующий проект
            p_actor_id := v_actor_id,
            p_contract_number := 'TEST-CONTRACT-002',
            p_role_description := 'Тестовая роль',
            p_salary_amount := 30000.00
        ) INTO v_contract_id;
```

```
        RAISE NOTICE '❌ ОШИБКА: Контракт не должен был создаться!';
    EXCEPTION
        WHEN OTHERS THEN
            v_error := SQLERRM;
            RAISE NOTICE '✅ Ожидаемая ошибка: %', v_error;
    END;
END $$;

RESET ROLE;

-- ====================================================
-- КЕЙС 3: Ошибка валидации - отрицательная зарплата (негативный)
-- ====================================================
\echo ''
\echo '=== КЕЙС 3: Ошибка валидации - отрицательная зарплата (негативный) ==='

SET ROLE studio_manager;

DO $$
DECLARE
    v_contract_id INTEGER;
    v_error TEXT;
    v_project_id INTEGER;
    v_actor_id INTEGER;
BEGIN
    -- Получаем ID из переменных сессии
    v_project_id := current_setting('app.test_project_id', true)::INTEGER;
    v_actor_id := current_setting('app.test_actor_id', true)::INTEGER;

    IF v_project_id IS NULL OR v_project_id = 0 OR v_actor_id IS NULL OR v_actor_id =
0 THEN
        RAISE WARNING '⚠️  В БД нет проектов или актёров. Пропускаем тест.';
        RETURN;
    END IF;

    BEGIN
        SELECT app.create_contract(
            p_project_id := v_project_id,
            p_actor_id := v_actor_id,
            p_contract_number := 'TEST-CONTRACT-003',
            p_role_description := 'Тестовая роль',
            p_salary_amount := -1000.00 -- Отрицательная зарплата
        ) INTO v_contract_id;

        RAISE NOTICE '❌ ОШИБКА: Контракт не должен был создаться!';
    EXCEPTION
        WHEN OTHERS THEN
            v_error := SQLERRM;
            RAISE NOTICE '✅ Ожидаемая ошибка: %', v_error;
    END;
END $$;

RESET ROLE;
```

```
-- ================================================
-- КЕЙС 4: Успешное обновление зарплаты (позитивный)
-- ================================================
\echo ''
\echo '=== КЕЙС 4: Успешное обновление зарплаты (позитивный) ==='

-- Получаем ID контракта (от имени postgres, который обходит RLS)
DO $$
DECLARE
    v_test_contract_id INTEGER;
BEGIN
    -- Получаем данные от имени текущего пользователя (postgres обходит RLS)
    SELECT contract_id INTO v_test_contract_id
    FROM app.contracts
    WHERE status = 'active'
    LIMIT 1;

    PERFORM set_config('app.test_contract_id', COALESCE(v_test_contract_id::TEXT,
'0'), false);
END $$;

SET ROLE studio_manager;

DO $$
DECLARE
    v_result BOOLEAN;
    v_contract_id INTEGER;
BEGIN
    -- Получаем ID контракта из переменных сессии
    v_contract_id := current_setting('app.test_contract_id', true)::INTEGER;

    IF v_contract_id IS NULL OR v_contract_id = 0 THEN
        RAISE WARNING '⚠️  В БД нет активных контрактов. Пропускаем тест обновления
зарплаты.';
        RETURN;
    END IF;

    -- Обновляем зарплату контракта
    SELECT app.update_contract_salary(
        p_contract_id := v_contract_id,
        p_new_salary_amount := 55000.00,
        p_reason := 'Повышение за хорошую работу'
    ) INTO v_result;

    IF v_result THEN
        RAISE NOTICE '✅ Зарплата успешно обновлена для контракта ID: %',
v_contract_id;
    ELSE
        RAISE NOTICE '❌ Ошибка обновления зарплаты';
    END IF;
END $$;
```

```
RESET ROLE;

-- ========================================================
-- КЕЙС 5: Попытка DDL неадмином (негативный)
-- ========================================================
\echo ''
\echo '=== КЕЙС 5: Попытка DDL неадмином (негативный) ==='

SET ROLE app_writer;

DO $$
BEGIN
    BEGIN
        -- Попытка создать таблицу (DDL операция)
        CREATE TABLE app.test_table (
            id INTEGER PRIMARY KEY
        );

        RAISE NOTICE '❌ ОШИБКА: Таблица не должна была создаться!';
        DROP TABLE IF EXISTS app.test_table;
    EXCEPTION
        WHEN insufficient_privilege THEN
            RAISE NOTICE '✅ Ожидаемая ошибка: Недостаточно прав для DDL операции';
        WHEN OTHERS THEN
            RAISE NOTICE '✅ Ожидаемая ошибка: %', SQLERRM;
    END;
END $$;

RESET ROLE;

-- ========================================================
-- КЕЙС 6: Попытка DML в audit (негативный)
-- ========================================================
\echo ''
\echo '=== КЕЙС 6: Попытка DML в audit (негативный) ==='

SET ROLE app_writer;

DO $$
BEGIN
    BEGIN
        -- Попытка вставить запись в audit.login_log
        INSERT INTO audit.login_log (username, is_successful)
        VALUES ('test_user', true);

        RAISE NOTICE '❌ ОШИБКА: Вставка в audit не должна была произойти!';
    EXCEPTION
        WHEN insufficient_privilege THEN
            RAISE NOTICE '✅ Ожидаемая ошибка: Недостаточно прав для DML в audit';
        WHEN OTHERS THEN
            RAISE NOTICE '✅ Ожидаемая ошибка: %', SQLERRM;
    END;
END $$;
```

```
RESET ROLE;

-- ====================================================
-- КЕЙС 7: Чтение разрешённых таблиц (позитивный)
-- ====================================================
\echo ''
\echo '=== КЕЙС 7: Чтение разрешённых таблиц (позитивный) ==='

-- Получаем ID актёра для теста (от имени postgres)
DO $$
DECLARE
    v_test_actor_id_for_read INTEGER;
BEGIN
    SELECT actor_id INTO v_test_actor_id_for_read FROM app.actors WHERE is_active =
true LIMIT 1;
    PERFORM set_config('app.test_actor_id_for_read',
COALESCE(v_test_actor_id_for_read::TEXT, '0'), false);
END $$;

SET ROLE app_reader;

DO $$
DECLARE
    v_count INTEGER;
    v_actor_id INTEGER;
BEGIN
    -- Устанавливаем current_actor_id для работы с RLS
    v_actor_id := current_setting('app.test_actor_id_for_read', true)::INTEGER;
    IF v_actor_id IS NOT NULL AND v_actor_id > 0 THEN
        PERFORM set_config('app.current_actor_id', v_actor_id::TEXT, false);
    END IF;

    -- Читаем таблицы, к которым есть доступ
    -- Для актёров RLS требует current_actor_id
    IF v_actor_id IS NOT NULL AND v_actor_id > 0 THEN
        SELECT COUNT(*) INTO v_count FROM app.actors;
        RAISE NOTICE '✅ Актёров доступно (с current_actor_id): %', v_count;
    ELSE
        RAISE NOTICE '⚠️  Актёры: требуется установка current_actor_id для RLS';
    END IF;

    -- Проекты доступны через политику get_current_user_studio()
    SELECT COUNT(*) INTO v_count FROM app.projects;
    RAISE NOTICE '✅ Проектов доступно: %', v_count;

    -- Справочники доступны без ограничений
    SELECT COUNT(*) INTO v_count FROM ref.languages;
    RAISE NOTICE '✅ Языков доступно: %', v_count;
END $$;

RESET ROLE;
```

```
-- ========================================================
-- КЕЙС 8: Вызов SECURITY DEFINER с невалидными данными (негативный)
-- ========================================================
\echo ''
\echo '=== КЕЙС 8: Вызов SECURITY DEFINER с невалидными данными (негативный) ==='

SET ROLE studio_manager;

DO $$
DECLARE
    v_session_id INTEGER;
    v_error TEXT;
    v_contract_id INTEGER;
BEGIN
    -- Получаем ID контракта из переменных сессии
    v_contract_id := current_setting('app.test_contract_id', true)::INTEGER;

    IF v_contract_id IS NULL OR v_contract_id = 0 THEN
        RAISE WARNING '⚠️  В БД нет активных контрактов. Пропускаем тест.';
        RETURN;
    END IF;

    BEGIN
        -- Пытаемся зарегистрировать сессию с невалидной длительностью
        SELECT app.register_recording_session(
            p_contract_id := v_contract_id,
            p_session_date := (CURRENT_DATE + INTERVAL '1 day')::DATE,
            p_session_time := '10:00:00'::TIME,
            p_studio_location := 'Студия A',
            p_duration_hours := 30.0 -- Невалидно: больше 24 часов
        ) INTO v_session_id;

        RAISE NOTICE '❌ ОШИБКА: Сессия не должна была создаться!';
    EXCEPTION
        WHEN OTHERS THEN
            v_error := SQLERRM;
            RAISE NOTICE '✅ Ожидаемая ошибка валидации: %', v_error;
    END;
END $$;

RESET ROLE;

-- ========================================================
-- КЕЙС 9: Вызов SECURITY DEFINER с валидными данными (позитивный)
-- ========================================================
\echo ''
\echo '=== КЕЙС 9: Вызов SECURITY DEFINER с валидными данными (позитивный) ==='

SET ROLE studio_manager;

DO $$
DECLARE
    v_session_id INTEGER;
```

```
    v_contract_id INTEGER;
BEGIN
    -- Получаем ID контракта из переменных сессии
    v_contract_id := current_setting('app.test_contract_id', true)::INTEGER;

    IF v_contract_id IS NULL OR v_contract_id = 0 THEN
        RAISE WARNING '⚠️  В БД нет активных контрактов. Пропускаем тест.';
        RETURN;
    END IF;

    -- Регистрируем сессию с валидными данными
    SELECT app.register_recording_session(
        p_contract_id := v_contract_id,
        p_session_date := (CURRENT_DATE + INTERVAL '1 day')::DATE,
        p_session_time := '10:00:00'::TIME,
        p_studio_location := 'Студия A, Москва',
        p_duration_hours := 4.5,
        p_notes := 'Тестовая сессия записи'
    ) INTO v_session_id;

    RAISE NOTICE '✅ Сессия успешно зарегистрирована с ID: % для контракта ID: %',
v_session_id, v_contract_id;
END $$;

RESET ROLE;

-- ========================================================
-- КЕЙС 10: Попытка обновления зарплаты в завершённом контракте (негативный)
-- ========================================================
\echo ''
\echo '=== КЕЙС 10: Попытка обновления зарплаты в завершённом контракте (негативный)
==='

SET ROLE studio_manager;

DO $$
DECLARE
    v_result BOOLEAN;
    v_error TEXT;
    v_project_id INTEGER;
    v_actor_id INTEGER;
    v_contract_id INTEGER;
BEGIN
    -- Получаем ID из переменных сессии
    v_project_id := current_setting('app.test_project_id', true)::INTEGER;
    v_actor_id := current_setting('app.test_actor_id', true)::INTEGER;

    IF v_project_id IS NULL OR v_project_id = 0 OR v_actor_id IS NULL OR v_actor_id =
0 THEN
        RAISE WARNING '⚠️  В БД нет проектов или актёров. Пропускаем тест.';
        RETURN;
    END IF;
```

```
    -- Сначала создаём и завершаем контракт для теста
    BEGIN
        -- Создаём тестовый контракт с уникальным номером
        SELECT app.create_contract(
            p_project_id := v_project_id,
            p_actor_id := v_actor_id,
            p_contract_number := 'TEST-COMPLETED-' || to_char(clock_timestamp(),
'YYYYMMDDHH24MISS'),
            p_role_description := 'Завершённый контракт для теста',
            p_salary_amount := 25000.00
        ) INTO v_contract_id;

        -- Завершаем контракт (от имени владельца)
        SET ROLE app_owner;
        UPDATE app.contracts
        SET status = 'completed', end_date = CURRENT_DATE
        WHERE contract_id = v_contract_id;
        RESET ROLE;

        -- Пытаемся обновить зарплату завершённого контракта
        BEGIN
            SELECT app.update_contract_salary(
                p_contract_id := v_contract_id,
                p_new_salary_amount := 30000.00
            ) INTO v_result;

            RAISE NOTICE '❌ ОШИБКА: Зарплата не должна была обновиться!';
        EXCEPTION
            WHEN OTHERS THEN
                v_error := SQLERRM;
                RAISE NOTICE '✅ Ожидаемая ошибка: %', v_error;
        END;
    EXCEPTION
        WHEN OTHERS THEN
            RAISE NOTICE '⚠️  Предупреждение: %', SQLERRM;
    END;
END $$;

RESET ROLE;

-- ========================================================
-- ИТОГОВЫЙ ОТЧЁТ
-- ========================================================
\echo ''
\echo '========================================================'
\echo 'ИТОГОВЫЙ ОТЧЁТ ПО ТЕСТАМ'
\echo '========================================================'

-- Подсчитываем успешные и неуспешные вызовы функций
SELECT
    'Всего вызовов функций' as metric,
    COUNT(*)::TEXT as value
FROM audit.function_calls
```

```
WHERE function_name IN ('create_contract', 'update_contract_salary',
'register_recording_session')
UNION ALL
SELECT
    'Успешных вызовов' as metric,
    COUNT(*)::TEXT as value
FROM audit.function_calls
WHERE function_name IN ('create_contract', 'update_contract_salary',
'register_recording_session')
  AND success = true
UNION ALL
SELECT
    'Неуспешных вызовов' as metric,
    COUNT(*)::TEXT as value
FROM audit.function_calls
WHERE function_name IN ('create_contract', 'update_contract_salary',
'register_recording_session')
  AND success = false;

\echo ''
\echo 'ПРОВЕРКА ЗАВЕРШЕНА!'
\echo '========================================================'
```

Приложение В

```sql
-- ======================================================
-- Сравнение CHECK vs триггер для бизнес-правил
-- ======================================================

-- БИЗНЕС-ПРАВИЛО:
-- Дата окончания контракта (end_date) должна быть >= даты подписания (signed_date)
-- И если контракт завершён (status = 'completed'), то end_date должна быть <=
CURRENT_DATE

-- ======================================================
-- ВАРИАНТ 1: CHECK CONSTRAINT
-- ======================================================

-- Создаём тестовую таблицу с CHECK
CREATE TABLE IF NOT EXISTS app.contracts_check_test (
    contract_id SERIAL PRIMARY KEY,
    contract_number VARCHAR(50) NOT NULL UNIQUE,
    signed_date DATE NOT NULL,
    end_date DATE,
    status VARCHAR(20) DEFAULT 'active',
    -- CHECK ограничение: end_date >= signed_date
    CONSTRAINT chk_end_after_start CHECK (
        end_date IS NULL OR end_date >= signed_date
    ),
    -- CHECK ограничение: если завершён, то end_date <= CURRENT_DATE
    CONSTRAINT chk_completed_end_date CHECK (
        status != 'completed' OR (end_date IS NOT NULL AND end_date <= CURRENT_DATE)
    )
);

COMMENT ON TABLE app.contracts_check_test IS 'Тестовая таблица для проверки CHECK
ограничений';

-- ======================================================
-- ВАРИАНТ 2: TRIGGER
-- ======================================================

-- Создаём тестовую таблицу без CHECK (будет проверка через триггер)
CREATE TABLE IF NOT EXISTS app.contracts_trigger_test (
    contract_id SERIAL PRIMARY KEY,
    contract_number VARCHAR(50) NOT NULL UNIQUE,
    signed_date DATE NOT NULL,
    end_date DATE,
    status VARCHAR(20) DEFAULT 'active'
);

COMMENT ON TABLE app.contracts_trigger_test IS 'Тестовая таблица для проверки
триггеров';

-- Функция триггера для валидации
CREATE OR REPLACE FUNCTION app.validate_contract_dates()
```

```
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    -- Проверка: end_date >= signed_date
    IF NEW.end_date IS NOT NULL AND NEW.end_date < NEW.signed_date THEN
        RAISE EXCEPTION 'ОШИБКА ВАЛИДАЦИИ: Дата окончания контракта (%) не может быть
раньше даты подписания (%)',
            NEW.end_date, NEW.signed_date;
    END IF;

    -- Проверка: если завершён, то end_date <= CURRENT_DATE
    IF NEW.status = 'completed' THEN
        IF NEW.end_date IS NULL THEN
            RAISE EXCEPTION 'ОШИБКА ВАЛИДАЦИИ: Завершённый контракт должен иметь дату
окончания';
        END IF;
        IF NEW.end_date > CURRENT_DATE THEN
            RAISE EXCEPTION 'ОШИБКА ВАЛИДАЦИИ: Дата окончания завершённого контракта
(%) не может быть в будущем',
                NEW.end_date;
        END IF;
    END IF;

    RETURN NEW;
END;
$$;

-- Создаём триггер
DROP TRIGGER IF EXISTS trg_validate_contract_dates ON app.contracts_trigger_test;
CREATE TRIGGER trg_validate_contract_dates
    BEFORE INSERT OR UPDATE ON app.contracts_trigger_test
    FOR EACH ROW
    EXECUTE FUNCTION app.validate_contract_dates();

COMMENT ON FUNCTION app.validate_contract_dates IS 'Триггерная функция для валидации
дат контракта';
COMMENT ON TRIGGER trg_validate_contract_dates ON app.contracts_trigger_test IS
'Триггер валидации дат контракта';

-- ====================================================
-- ТЕСТИРОВАНИЕ ПРОИЗВОДИТЕЛЬНОСТИ
-- ====================================================

-- Функция для генерации тестовых данных
CREATE OR REPLACE FUNCTION app.generate_test_contracts(
    p_count INTEGER,
    p_table_name VARCHAR
)
RETURNS TABLE (
    inserted_count INTEGER,
    failed_count INTEGER,
    total_time_ms NUMERIC
```

```
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_start_time TIMESTAMPTZ;
    v_end_time TIMESTAMPTZ;
    v_execution_time NUMERIC;
    v_inserted INTEGER := 0;
    v_failed INTEGER := 0;
    v_i INTEGER;
    v_signed_date DATE;
    v_end_date DATE;
    v_status VARCHAR(20);
BEGIN
    v_start_time := clock_timestamp();

    FOR v_i IN 1..p_count LOOP
        BEGIN
            -- Генерируем случайные, но валидные данные
            v_signed_date := CURRENT_DATE - (random() * 365)::INTEGER;
            v_end_date := v_signed_date + (random() * 180)::INTEGER;

            IF random() > 0.7 THEN
                v_status := 'completed';
                -- Для завершённых контрактов end_date должна быть в прошлом
                IF v_end_date > CURRENT_DATE THEN
                    v_end_date := CURRENT_DATE - (random() * 30)::INTEGER;
                END IF;
            ELSE
                v_status := 'active';
            END IF;

            -- Вставляем в соответствующую таблицу
            IF p_table_name = 'check' THEN
                INSERT INTO app.contracts_check_test (
                    contract_number,
                    signed_date,
                    end_date,
                    status
                ) VALUES (
                    'TEST-CHECK-' || v_i,
                    v_signed_date,
                    v_end_date,
                    v_status
                );
            ELSIF p_table_name = 'trigger' THEN
                INSERT INTO app.contracts_trigger_test (
                    contract_number,
                    signed_date,
                    end_date,
                    status
                ) VALUES (
                    'TEST-TRIGGER-' || v_i,
```

```
                    v_signed_date,
                    v_end_date,
                    v_status
                );
            END IF;

            v_inserted := v_inserted + 1;

        EXCEPTION
            WHEN OTHERS THEN
                v_failed := v_failed + 1;
                -- Игнорируем ошибки и продолжаем
        END;
    END LOOP;

    v_end_time := clock_timestamp();
    v_execution_time := EXTRACT(EPOCH FROM (v_end_time - v_start_time)) * 1000;

    RETURN QUERY SELECT v_inserted, v_failed, v_execution_time;
END;
$$;

-- Функция для тестирования производительности
CREATE OR REPLACE FUNCTION app.test_performance_check_vs_trigger(
    p_test_count INTEGER DEFAULT 10000
)
RETURNS TABLE (
    method VARCHAR,
    inserted_count INTEGER,
    failed_count INTEGER,
    total_time_ms NUMERIC,
    avg_time_per_insert_ms NUMERIC
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_check_result RECORD;
    v_trigger_result RECORD;
BEGIN
    -- Очищаем тестовые таблицы
    TRUNCATE TABLE app.contracts_check_test;
    TRUNCATE TABLE app.contracts_trigger_test;

    -- Тест CHECK
    SELECT * INTO v_check_result
    FROM app.generate_test_contracts(p_test_count, 'check');

    -- Тест TRIGGER
    SELECT * INTO v_trigger_result
    FROM app.generate_test_contracts(p_test_count, 'trigger');

    -- Возвращаем результаты
    RETURN QUERY
```

```
    SELECT
        'CHECK'::VARCHAR as method,
        v_check_result.inserted_count,
        v_check_result.failed_count,
        v_check_result.total_time_ms,
        CASE
            WHEN v_check_result.inserted_count > 0
            THEN v_check_result.total_time_ms / v_check_result.inserted_count
            ELSE 0
        END as avg_time_per_insert_ms
    UNION ALL
    SELECT
        'TRIGGER'::VARCHAR as method,
        v_trigger_result.inserted_count,
        v_trigger_result.failed_count,
        v_trigger_result.total_time_ms,
        CASE
            WHEN v_trigger_result.inserted_count > 0
            THEN v_trigger_result.total_time_ms / v_trigger_result.inserted_count
            ELSE 0
        END as avg_time_per_insert_ms;
END;
$$;

-- Даём права на тестовые таблицы
GRANT SELECT, INSERT ON app.contracts_check_test TO app_owner;
GRANT SELECT, INSERT ON app.contracts_trigger_test TO app_owner;
GRANT EXECUTE ON FUNCTION app.generate_test_contracts TO app_owner;
GRANT EXECUTE ON FUNCTION app.test_performance_check_vs_trigger TO app_owner;

COMMENT ON FUNCTION app.generate_test_contracts IS 'Генерация тестовых данных для
сравнения CHECK vs триггер';
COMMENT ON FUNCTION app.test_performance_check_vs_trigger IS 'Тест производительности
CHECK vs триггер на заданном количестве строк';

-- Уведомление
DO $$
BEGIN
    RAISE NOTICE '✅ Тестовые таблицы и функции для сравнения CHECK vs триггер
созданы';
    RAISE NOTICE '   Для запуска теста выполните: SELECT * FROM
app.test_performance_check_vs_trigger(10000);';
END $$;
```

```
-- ====================================================
-- Тест производительности CHECK vs триггер
-- ====================================================

\echo '====================================================='
\echo 'ТЕСТ ПРОИЗВОДИТЕЛЬНОСТИ: CHECK vs TRIGGER'
```

```
\echo '=================================================='

-- Запускаем тест на 10,000 строк
\echo ''
\echo 'Запуск теста на 10,000 строк...'
\echo 'Это может занять некоторое время...'

SELECT * FROM app.test_performance_check_vs_trigger(10000);

\echo ''
\echo '=================================================='
\echo 'СТАТИСТИКА'
\echo '=================================================='

-- Показываем количество записей в каждой таблице
SELECT
    'CHECK метод' as method,
    COUNT(*) as total_records
FROM app.contracts_check_test
UNION ALL
SELECT
    'TRIGGER метод' as method,
    COUNT(*) as total_records
FROM app.contracts_trigger_test;

-- Показываем примеры записей
\echo ''
\echo 'Примеры записей из CHECK таблицы:'
SELECT * FROM app.contracts_check_test LIMIT 5;

\echo ''
\echo 'Примеры записей из TRIGGER таблицы:'
SELECT * FROM app.contracts_trigger_test LIMIT 5;

\echo ''
\echo '=================================================='
\echo 'ПРОВЕРКА ВАЛИДАЦИИ'
\echo '=================================================='

-- Проверяем, что все записи валидны
SELECT
    'CHECK: Валидных записей' as check_type,
    COUNT(*) as valid_count
FROM app.contracts_check_test
WHERE (end_date IS NULL OR end_date >= signed_date)
  AND (status != 'completed' OR (end_date IS NOT NULL AND end_date <= CURRENT_DATE))
UNION ALL
SELECT
    'CHECK: Всего записей' as check_type,
    COUNT(*) as total_count
FROM app.contracts_check_test;

SELECT
```

```
    'TRIGGER: Всего записей' as check_type,
    COUNT(*) as total_count
FROM app.contracts_trigger_test;

\echo ''
\echo '=================================================='
\echo 'Сравнение CHECK и TRIGGER'
\echo '=================================================='

-- Сохраняем результаты один раз для использования
WITH performance_results AS (
    SELECT * FROM app.test_performance_check_vs_trigger(10000)
),
check_data AS (
    SELECT * FROM performance_results WHERE method = 'CHECK'
),
trigger_data AS (
    SELECT * FROM performance_results WHERE method = 'TRIGGER'
),
comparison_table AS (
    SELECT
        1 as sort_order,
        'Успешно вставлено' as "Параметр",
        c.inserted_count::TEXT as "CHECK",
        t.inserted_count::TEXT as "TRIGGER",
        CASE
            WHEN c.inserted_count > t.inserted_count THEN
                '+' || (c.inserted_count - t.inserted_count)::TEXT
            ELSE
                (c.inserted_count - t.inserted_count)::TEXT
        END as "Разница"
    FROM check_data c, trigger_data t
    UNION ALL
    SELECT
        2,
        'Неудачных вставок',
        c.failed_count::TEXT,
        t.failed_count::TEXT,
        CASE
            WHEN c.failed_count > t.failed_count THEN
                '+' || (c.failed_count - t.failed_count)::TEXT
            ELSE
                (c.failed_count - t.failed_count)::TEXT
        END
    FROM check_data c, trigger_data t
    UNION ALL
    SELECT
        3,
        'Общее время (мс)',
        ROUND(c.total_time_ms, 2)::TEXT,
        ROUND(t.total_time_ms, 2)::TEXT,
        CASE
            WHEN c.total_time_ms > t.total_time_ms THEN
```

```
                '+' || ROUND(c.total_time_ms - t.total_time_ms, 2)::TEXT || ' мс (' ||
                ROUND((c.total_time_ms - t.total_time_ms) / c.total_time_ms * 100,
1)::TEXT || '%)'
            ELSE
                ROUND(c.total_time_ms - t.total_time_ms, 2)::TEXT || ' мс (' ||
                ROUND(ABS(c.total_time_ms - t.total_time_ms) / t.total_time_ms * 100,
1)::TEXT || '%)'
        END
    FROM check_data c, trigger_data t
    UNION ALL
    SELECT
        4,
        'Среднее время на вставку (мс)',
        ROUND(c.avg_time_per_insert_ms, 5)::TEXT,
        ROUND(t.avg_time_per_insert_ms, 5)::TEXT,
        CASE
            WHEN c.avg_time_per_insert_ms > t.avg_time_per_insert_ms THEN
                '+' || ROUND(c.avg_time_per_insert_ms - t.avg_time_per_insert_ms,
5)::TEXT || ' мс'
            ELSE
                ROUND(c.avg_time_per_insert_ms - t.avg_time_per_insert_ms, 5)::TEXT ||
' мс'
        END
    FROM check_data c, trigger_data t
)
SELECT "Параметр", "CHECK", "TRIGGER", "Разница"
FROM comparison_table
ORDER BY sort_order;

\echo ''
\echo '============================================================'
\echo 'ПРОВЕРКА ВАЛИДАЦИИ'
\echo '============================================================'
\echo ''
\echo 'Попытка вставить невалидные данные для проверки...'

-- Тест CHECK: попытка вставить невалидные данные
DO $$
BEGIN
    BEGIN
        INSERT INTO app.contracts_check_test (
            contract_number,
            signed_date,
            end_date,
            status
        ) VALUES (
            'TEST-INVALID-001',
            CURRENT_DATE,
            CURRENT_DATE - INTERVAL '1 day', -- Невалидно: end_date < signed_date
            'active'
        );

        RAISE NOTICE '❌ ОШИБКА: CHECK не сработал!';
```

```
    EXCEPTION
        WHEN check_violation THEN
            RAISE NOTICE '✅ CHECK сработал корректно: %', SQLERRM;
        WHEN OTHERS THEN
            RAISE NOTICE '⚠️  Неожиданная ошибка: %', SQLERRM;
    END;
END $$;

-- Тест TRIGGER: попытка вставить невалидные данные
DO $$
BEGIN
    BEGIN
        INSERT INTO app.contracts_trigger_test (
            contract_number,
            signed_date,
            end_date,
            status
        ) VALUES (
            'TEST-INVALID-002',
            CURRENT_DATE,
            CURRENT_DATE - INTERVAL '1 day', -- Невалидно: end_date < signed_date
            'active'
        );

        RAISE NOTICE '❌ ОШИБКА: TRIGGER не сработал!';
    EXCEPTION
        WHEN OTHERS THEN
            RAISE NOTICE '✅ TRIGGER сработал корректно: %', SQLERRM;
    END;
END $$;

\echo ''
\echo '=================================================='
```

Приложение Г

```
-- ====================================================
-- Создание таблицы журнала вызовов функций
-- ====================================================

-- Создаём таблицу для журнала вызовов SECURITY DEFINER-функций
CREATE TABLE IF NOT EXISTS audit.function_calls (
    call_id BIGSERIAL PRIMARY KEY,
    call_time TIMESTAMPTZ DEFAULT NOW() NOT NULL,
    function_name VARCHAR(255) NOT NULL,
    function_schema VARCHAR(100) NOT NULL,
    caller_role VARCHAR(100) NOT NULL,
    caller_username VARCHAR(100) NOT NULL,
    input_params JSONB,
    return_value JSONB,
    success BOOLEAN NOT NULL DEFAULT true,
    error_message TEXT,
    execution_time_ms NUMERIC(10, 3),
    client_ip INET,
    created_at TIMESTAMPTZ DEFAULT NOW() NOT NULL
);

-- Создаём индексы для быстрого поиска
CREATE INDEX IF NOT EXISTS idx_function_calls_time ON audit.function_calls(call_time);
CREATE INDEX IF NOT EXISTS idx_function_calls_function ON
audit.function_calls(function_schema, function_name);
CREATE INDEX IF NOT EXISTS idx_function_calls_caller ON
audit.function_calls(caller_username);
CREATE INDEX IF NOT EXISTS idx_function_calls_success ON audit.function_calls(success)
WHERE success = false;

-- Комментарии
COMMENT ON TABLE audit.function_calls IS 'Журнал вызовов SECURITY DEFINER-функций для
аудита безопасности';
COMMENT ON COLUMN audit.function_calls.call_id IS 'Уникальный идентификатор записи';
COMMENT ON COLUMN audit.function_calls.call_time IS 'Время вызова функции';
COMMENT ON COLUMN audit.function_calls.function_name IS 'Имя вызванной функции';
COMMENT ON COLUMN audit.function_calls.function_schema IS 'Схема функции';
COMMENT ON COLUMN audit.function_calls.caller_role IS 'Роль, от имени которой вызвана
функция';
COMMENT ON COLUMN audit.function_calls.caller_username IS 'Имя пользователя,
вызвавшего функцию';
COMMENT ON COLUMN audit.function_calls.input_params IS 'Параметры вызова в формате
JSON';
COMMENT ON COLUMN audit.function_calls.return_value IS 'Возвращаемое значение (если
есть)';
COMMENT ON COLUMN audit.function_calls.success IS 'Успешность выполнения функции';
COMMENT ON COLUMN audit.function_calls.error_message IS 'Сообщение об ошибке (если
была)';
COMMENT ON COLUMN audit.function_calls.execution_time_ms IS 'Время выполнения в
миллисекундах';
COMMENT ON COLUMN audit.function_calls.client_ip IS 'IP-адрес клиента';
```

```sql
-- Запрещаем прямой доступ к таблице (только через функции)
REVOKE ALL ON audit.function_calls FROM PUBLIC;
REVOKE INSERT, UPDATE, DELETE ON audit.function_calls FROM PUBLIC;

-- Разрешаем чтение только auditor
GRANT SELECT ON audit.function_calls TO auditor;

-- Создаём роль для логирования (только INSERT)
DO $$
BEGIN
    IF NOT EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'function_logger') THEN
        CREATE ROLE function_logger WITH NOLOGIN;
    END IF;
END $$;

GRANT USAGE ON SCHEMA audit TO function_logger;
GRANT INSERT ON audit.function_calls TO function_logger;
GRANT USAGE ON SEQUENCE audit.function_calls_call_id_seq TO function_logger;

-- Функция-хелпер для логирования вызова функции
CREATE OR REPLACE FUNCTION audit.log_function_call(
    p_function_schema VARCHAR,
    p_function_name VARCHAR,
    p_input_params JSONB DEFAULT NULL,
    p_return_value JSONB DEFAULT NULL,
    p_success BOOLEAN DEFAULT true,
    p_error_message TEXT DEFAULT NULL,
    p_execution_time_ms NUMERIC DEFAULT NULL
)
RETURNS void
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = audit, public
AS $$
BEGIN
    INSERT INTO audit.function_calls (
        function_schema,
        function_name,
        caller_role,
        caller_username,
        input_params,
        return_value,
        success,
        error_message,
        execution_time_ms,
        client_ip
    ) VALUES (
        p_function_schema,
        p_function_name,
        COALESCE(current_setting('role', true), session_user)::VARCHAR,
        session_user::VARCHAR(100),
        p_input_params,
```

```
        p_return_value,
        p_success,
        p_error_message,
        p_execution_time_ms,
        inet_client_addr()
    );
EXCEPTION
    WHEN OTHERS THEN
        -- В случае ошибки логирования просто игнорируем (чтобы не ломать основную
функцию)
        NULL;
END;
$$;

-- Даём доступ к функции логирования
GRANT EXECUTE ON FUNCTION audit.log_function_call TO function_logger;
GRANT function_logger TO app_owner;

COMMENT ON FUNCTION audit.log_function_call IS 'Вспомогательная функция для
логирования вызовов SECURITY DEFINER-функций';

-- Уведомление
DO $$
BEGIN
    RAISE NOTICE '✅ Таблица audit.function_calls и функция логирования успешно
созданы';
END $$;
```

```
-- ======================================================
-- SECURITY DEFINER-функции для чувствительных операций
-- ======================================================

-- Устанавливаем роль для создания функций
SET ROLE app_owner;

-- ======================================================
-- ФУНКЦИЯ 1: create_contract
-- Создание контракта с актёром (чувствительная операция)
-- ======================================================

CREATE OR REPLACE FUNCTION app.create_contract(
    p_project_id INTEGER,
    p_actor_id INTEGER,
    p_contract_number VARCHAR(50),
    p_role_description TEXT,
    p_salary_amount DECIMAL(10,2),
    p_currency VARCHAR(3) DEFAULT 'USD',
    p_terms TEXT DEFAULT NULL,
    p_signed_date DATE DEFAULT CURRENT_DATE,
    p_end_date DATE DEFAULT NULL
)
```

```
RETURNS INTEGER
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = app, ref, audit, public
AS $$
DECLARE
    v_start_time TIMESTAMPTZ;
    v_end_time TIMESTAMPTZ;
    v_execution_time NUMERIC;
    v_contract_id INTEGER;
    v_input_params JSONB;
    v_return_value JSONB;
BEGIN
    -- Засекаем время выполнения
    v_start_time := clock_timestamp();

    -- Формируем JSON с параметрами для логирования
    v_input_params := jsonb_build_object(
        'project_id', p_project_id,
        'actor_id', p_actor_id,
        'contract_number', p_contract_number,
        'role_description', p_role_description,
        'salary_amount', p_salary_amount,
        'currency', p_currency
    );

    -- ВАЛИДАЦИЯ ВХОДНЫХ ДАННЫХ

    -- Проверка существования проекта
    IF NOT EXISTS (SELECT 1 FROM app.projects WHERE project_id = p_project_id) THEN
        PERFORM audit.log_function_call(
            'app', 'create_contract',
            v_input_params,
            NULL,
            false,
            'ОШИБКА: Проект с ID ' || p_project_id || ' не существует',
            NULL
        );
        RAISE EXCEPTION 'Проект с ID % не существует', p_project_id;
    END IF;

    -- Проверка существования актёра
    IF NOT EXISTS (SELECT 1 FROM app.actors WHERE actor_id = p_actor_id AND is_active
= true) THEN
        PERFORM audit.log_function_call(
            'app', 'create_contract',
            v_input_params,
            NULL,
            false,
            'ОШИБКА: Актёр с ID ' || p_actor_id || ' не существует или неактивен',
            NULL
        );
        RAISE EXCEPTION 'Актёр с ID % не существует или неактивен', p_actor_id;
```

```
    END IF;

    -- Проверка уникальности номера контракта
    IF EXISTS (SELECT 1 FROM app.contracts WHERE contract_number = p_contract_number)
THEN
        PERFORM audit.log_function_call(
            'app', 'create_contract',
            v_input_params,
            NULL,
            false,
            'ОШИБКА: Контракт с номером ' || p_contract_number || ' уже существует',
            NULL
        );
        RAISE EXCEPTION 'Контракт с номером % уже существует', p_contract_number;
    END IF;

    -- Проверка суммы контракта
    IF p_salary_amount IS NULL OR p_salary_amount <= 0 THEN
        PERFORM audit.log_function_call(
            'app', 'create_contract',
            v_input_params,
            NULL,
            false,
            'ОШИБКА: Сумма контракта должна быть положительным числом',
            NULL
        );
        RAISE EXCEPTION 'Сумма контракта должна быть положительным числом';
    END IF;

    -- Проверка дат (end_date должен быть после signed_date)
    IF p_end_date IS NOT NULL AND p_signed_date IS NOT NULL AND p_end_date <
p_signed_date THEN
        PERFORM audit.log_function_call(
            'app', 'create_contract',
            v_input_params,
            NULL,
            false,
            'ОШИБКА: Дата окончания контракта не может быть раньше даты подписания',
            NULL
        );
        RAISE EXCEPTION 'Дата окончания контракта не может быть раньше даты
подписания';
    END IF;

    -- Проверка обязательных полей
    IF p_contract_number IS NULL OR trim(p_contract_number) = '' THEN
        PERFORM audit.log_function_call(
            'app', 'create_contract',
            v_input_params,
            NULL,
            false,
            'ОШИБКА: Номер контракта обязателен для заполнения',
            NULL
```

```
        );
        RAISE EXCEPTION 'Номер контракта обязателен для заполнения';
    END IF;

    IF p_role_description IS NULL OR trim(p_role_description) = '' THEN
        PERFORM audit.log_function_call(
            'app', 'create_contract',
            v_input_params,
            NULL,
            false,
            'ОШИБКА: Описание роли обязательно для заполнения',
            NULL
        );
        RAISE EXCEPTION 'Описание роли обязательно для заполнения';
    END IF;

    -- СОЗДАНИЕ КОНТРАКТА
    INSERT INTO app.contracts (
        project_id,
        actor_id,
        contract_number,
        role_description,
        salary_amount,
        currency,
        terms,
        signed_date,
        end_date,
        status
    ) VALUES (
        p_project_id,
        p_actor_id,
        p_contract_number,
        p_role_description,
        p_salary_amount,
        p_currency,
        p_terms,
        p_signed_date,
        p_end_date,
        'active'
    )
    RETURNING contract_id INTO v_contract_id;

    -- Формируем возвращаемое значение для логирования
    v_return_value := jsonb_build_object('contract_id', v_contract_id);

    -- Вычисляем время выполнения
    v_end_time := clock_timestamp();
    v_execution_time := EXTRACT(EPOCH FROM (v_end_time - v_start_time)) * 1000;

    -- ЛОГИРОВАНИЕ УСПЕШНОГО ВЫЗОВА
    PERFORM audit.log_function_call(
        'app', 'create_contract',
        v_input_params,
```

```
        v_return_value,
        true,
        NULL,
        v_execution_time
    );

    RETURN v_contract_id;

EXCEPTION
    WHEN OTHERS THEN
        -- Вычисляем время выполнения при ошибке
        v_end_time := clock_timestamp();
        v_execution_time := EXTRACT(EPOCH FROM (v_end_time - v_start_time)) * 1000;

        -- ЛОГИРОВАНИЕ ОШИБКИ
        PERFORM audit.log_function_call(
            'app', 'create_contract',
            v_input_params,
            NULL,
            false,
            'ОШИБКА: ' || SQLERRM,
            v_execution_time
        );

        -- Пробрасываем ошибку дальше
        RAISE;
END;
$$;

-- ======================================================
-- ФУНКЦИЯ 2: update_contract_salary
-- Обновление зарплаты в контракте (конфиденциальные данные)
-- ======================================================

CREATE OR REPLACE FUNCTION app.update_contract_salary(
    p_contract_id INTEGER,
    p_new_salary_amount DECIMAL(10,2),
    p_reason TEXT DEFAULT NULL
)
RETURNS BOOLEAN
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = app, ref, audit, public
AS $$
DECLARE
    v_start_time TIMESTAMPTZ;
    v_end_time TIMESTAMPTZ;
    v_execution_time NUMERIC;
    v_old_salary DECIMAL(10,2);
    v_input_params JSONB;
    v_return_value JSONB;
BEGIN
    v_start_time := clock_timestamp();
```

```
    -- Формируем JSON с параметрами
    v_input_params := jsonb_build_object(
        'contract_id', p_contract_id,
        'new_salary_amount', p_new_salary_amount,
        'reason', p_reason
    );

    -- ВАЛИДАЦИЯ

    -- Проверка существования контракта
    SELECT salary_amount INTO v_old_salary
    FROM app.contracts
    WHERE contract_id = p_contract_id;

    IF NOT FOUND THEN
        PERFORM audit.log_function_call(
            'app', 'update_contract_salary',
            v_input_params,
            NULL,
            false,
            'ОШИБКА: Контракт с ID ' || p_contract_id || ' не найден',
            NULL
        );
        RAISE EXCEPTION 'Контракт с ID % не найден', p_contract_id;
    END IF;

    -- Проверка суммы
    IF p_new_salary_amount IS NULL OR p_new_salary_amount <= 0 THEN
        PERFORM audit.log_function_call(
            'app', 'update_contract_salary',
            v_input_params,
            NULL,
            false,
            'ОШИБКА: Новая сумма должна быть положительным числом',
            NULL
        );
        RAISE EXCEPTION 'Новая сумма должна быть положительным числом';
    END IF;

    -- Проверка, что сумма изменилась
    IF p_new_salary_amount = v_old_salary THEN
        PERFORM audit.log_function_call(
            'app', 'update_contract_salary',
            v_input_params,
            NULL,
            false,
            'ОШИБКА: Новая сумма совпадает со старой',
            NULL
        );
        RAISE EXCEPTION 'Новая сумма совпадает со старой (%.2f)', v_old_salary;
    END IF;
```

```
    -- Проверка статуса контракта
    IF EXISTS (
        SELECT 1 FROM app.contracts
        WHERE contract_id = p_contract_id
        AND status IN ('cancelled', 'completed')
    ) THEN
        PERFORM audit.log_function_call(
            'app', 'update_contract_salary',
            v_input_params,
            NULL,
            false,
            'ОШИБКА: Нельзя изменить зарплату в завершённом или отменённом контракте',
            NULL
        );
        RAISE EXCEPTION 'Нельзя изменить зарплату в завершённом или отменённом
контракте';
    END IF;

    -- ОБНОВЛЕНИЕ ЗАРПЛАТЫ
    UPDATE app.contracts
    SET
        salary_amount = p_new_salary_amount
    WHERE contract_id = p_contract_id;

    -- Формируем возвращаемое значение
    v_return_value := jsonb_build_object(
        'contract_id', p_contract_id,
        'old_salary', v_old_salary,
        'new_salary', p_new_salary_amount,
        'updated', true
    );

    -- Вычисляем время выполнения
    v_end_time := clock_timestamp();
    v_execution_time := EXTRACT(EPOCH FROM (v_end_time - v_start_time)) * 1000;

    -- ЛОГИРОВАНИЕ
    PERFORM audit.log_function_call(
        'app', 'update_contract_salary',
        v_input_params,
        v_return_value,
        true,
        NULL,
        v_execution_time
    );

    RETURN true;

EXCEPTION
    WHEN OTHERS THEN
        v_end_time := clock_timestamp();
        v_execution_time := EXTRACT(EPOCH FROM (v_end_time - v_start_time)) * 1000;
```

```
        PERFORM audit.log_function_call(
            'app', 'update_contract_salary',
            v_input_params,
            NULL,
            false,
            'ОШИБКА: ' || SQLERRM,
            v_execution_time
        );

        RAISE;
END;
$$;

-- ========================================================
-- ФУНКЦИЯ 3: register_recording_session
-- Регистрация сессии записи (бизнес-транзакция)
-- ========================================================

CREATE OR REPLACE FUNCTION app.register_recording_session(
    p_contract_id INTEGER,
    p_session_date DATE,
    p_session_time TIME,
    p_studio_location VARCHAR(100),
    p_duration_hours DECIMAL(4,2),
    p_notes TEXT DEFAULT NULL
)
RETURNS INTEGER
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = app, ref, audit, public
AS $$
DECLARE
    v_start_time TIMESTAMPTZ;
    v_end_time TIMESTAMPTZ;
    v_execution_time NUMERIC;
    v_session_id INTEGER;
    v_contract_status VARCHAR(20);
    v_input_params JSONB;
    v_return_value JSONB;
BEGIN
    v_start_time := clock_timestamp();

    -- Формируем JSON с параметрами
    v_input_params := jsonb_build_object(
        'contract_id', p_contract_id,
        'session_date', p_session_date,
        'session_time', p_session_time,
        'studio_location', p_studio_location,
        'duration_hours', p_duration_hours
    );

    -- ВАЛИДАЦИЯ
```

```
    -- Проверка существования контракта
    SELECT status INTO v_contract_status
    FROM app.contracts
    WHERE contract_id = p_contract_id;

    IF NOT FOUND THEN
        PERFORM audit.log_function_call(
            'app', 'register_recording_session',
            v_input_params,
            NULL,
            false,
            'ОШИБКА: Контракт с ID ' || p_contract_id || ' не найден',
            NULL
        );
        RAISE EXCEPTION 'Контракт с ID % не найден', p_contract_id;
    END IF;

    -- Проверка статуса контракта
    IF v_contract_status != 'active' THEN
        PERFORM audit.log_function_call(
            'app', 'register_recording_session',
            v_input_params,
            NULL,
            false,
            'ОШИБКА: Контракт должен быть активным (текущий статус: ' ||
v_contract_status || ')',
            NULL
        );
        RAISE EXCEPTION 'Контракт должен быть активным (текущий статус: %)',
v_contract_status;
    END IF;

    -- Проверка даты сессии (не должна быть в прошлом более чем на 1 день)
    IF p_session_date < CURRENT_DATE - INTERVAL '1 day' THEN
        PERFORM audit.log_function_call(
            'app', 'register_recording_session',
            v_input_params,
            NULL,
            false,
            'ОШИБКА: Дата сессии не может быть в прошлом более чем на 1 день',
            NULL
        );
        RAISE EXCEPTION 'Дата сессии не может быть в прошлом более чем на 1 день';
    END IF;

    -- Проверка длительности
    IF p_duration_hours IS NULL OR p_duration_hours <= 0 OR p_duration_hours > 24 THEN
        PERFORM audit.log_function_call(
            'app', 'register_recording_session',
            v_input_params,
            NULL,
            false,
            'ОШИБКА: Длительность сессии должна быть от 0 до 24 часов',
```

```
            NULL
        );
        RAISE EXCEPTION 'Длительность сессии должна быть от 0 до 24 часов';
    END IF;

    -- Проверка обязательных полей
    IF p_studio_location IS NULL OR trim(p_studio_location) = '' THEN
        PERFORM audit.log_function_call(
            'app', 'register_recording_session',
            v_input_params,
            NULL,
            false,
            'ОШИБКА: Место проведения сессии обязательно для заполнения',
            NULL
        );
        RAISE EXCEPTION 'Место проведения сессии обязательно для заполнения';
    END IF;

    -- СОЗДАНИЕ СЕССИИ
    INSERT INTO app.recording_sessions (
        contract_id,
        session_date,
        session_time,
        studio_location,
        duration_hours,
        notes,
        status
    ) VALUES (
        p_contract_id,
        p_session_date,
        p_session_time,
        p_studio_location,
        p_duration_hours,
        p_notes,
        'scheduled'
    )
    RETURNING session_id INTO v_session_id;

    -- Формируем возвращаемое значение
    v_return_value := jsonb_build_object(
        'session_id', v_session_id,
        'contract_id', p_contract_id,
        'session_date', p_session_date,
        'status', 'scheduled'
    );

    -- Вычисляем время выполнения
    v_end_time := clock_timestamp();
    v_execution_time := EXTRACT(EPOCH FROM (v_end_time - v_start_time)) * 1000;

    -- ЛОГИРОВАНИЕ
    PERFORM audit.log_function_call(
        'app', 'register_recording_session',
```

```
        v_input_params,
        v_return_value,
        true,
        NULL,
        v_execution_time
    );

    RETURN v_session_id;

EXCEPTION
    WHEN OTHERS THEN
        v_end_time := clock_timestamp();
        v_execution_time := EXTRACT(EPOCH FROM (v_end_time - v_start_time)) * 1000;

        PERFORM audit.log_function_call(
            'app', 'register_recording_session',
            v_input_params,
            NULL,
            false,
            'ОШИБКА: ' || SQLERRM,
            v_execution_time
        );

        RAISE;
END;
$$;

-- Сбрасываем роль
RESET ROLE;

-- Даём доступ к функциям нужным ролям
GRANT EXECUTE ON FUNCTION app.create_contract TO app_writer, studio_manager;
GRANT EXECUTE ON FUNCTION app.update_contract_salary TO app_writer, studio_manager;
GRANT EXECUTE ON FUNCTION app.register_recording_session TO app_writer,
studio_manager;

-- Комментарии к функциям
COMMENT ON FUNCTION app.create_contract IS 'SECURITY DEFINER: Создание контракта с
актёром. Валидирует входные данные и логирует операцию';
COMMENT ON FUNCTION app.update_contract_salary IS 'SECURITY DEFINER: Обновление
зарплаты в контракте. Проверяет права и статус контракта';
COMMENT ON FUNCTION app.register_recording_session IS 'SECURITY DEFINER: Регистрация
сессии записи. Валидирует контракт и параметры сессии';

-- Уведомление
DO $$
BEGIN
    RAISE NOTICE '✅ SECURITY DEFINER-функции успешно созданы:';
    RAISE NOTICE '   - app.create_contract';
    RAISE NOTICE '   - app.update_contract_salary';
    RAISE NOTICE '   - app.register_recording_session';
END $$;
```